Bash scripts
from Learn Linux TV

By: Gabriel Merino

July 12, 2026

Contents

Chapter 1

Hello World

1.1 To create a bash script

First, we'll have to create a file with a .sh extention. Then we can edit it in with whatever command we want.

1.1: 1-bash.sh

```
ls
pwd
```

The way bash scripts are execute is top to bottom.

1.2 how to run it?

First we'll have to change the script properties from the terminal.

1.2: Terminal

```
sudo chmod +x 1-bash.sh
```

This makes the script executable. Then we just have to execute it from the terminal:

1.3: Terminal

```
./1-bash.sh
```

1.3 Good practice for new scripts

What we are using here is a "shebang". This tells the interpreter whether they should be the ones executing this script. Otherwise, they should call the specified interpreter to execute the script.

1.4: 1-bash.sh

```
#!/bin/bash

ls
pwd
```

1.4 Comments

You can add comments to you script using '#' followed by text.

1.5 Hello World!

So to write anything out, you use the echo command. Here we'll do what we always do at the beginning and write our hello world.

1.5: 1-bash.sh

```
#!/bin/bash

echo "Hello World"
```

Chapter 2

Variables

2.1 Definition and usage

defining a variable is completely different than using it. To use it, you have to reference the variable simlar to C.

2.1: 2-bash.sh

```
#!/bin/bash

# Do not use spaces between the variable, '=' and value
myName="John Doe"

echo $myName
```

2.2 Variables are temporary

This type of variable definition would not survive longer than the session. What I mean by session is that once you close the terminal or logout that specific user, the variables are going to be erased. This means that defining the variables using scripts can garantee that needed variables are going to be available for the script itself.

2.3 String interpolation available

If you know about string interpolation, you know the dollar sign is the standard way to reference a variable inside a string and this mechanism also works in bash.

2.2: 2-bash.sh

```
#!/bin/bash

myName="John"
myAge="99"

echo "Hello, my name is $myName and I'm $myAge years old"
```

Also, it is required to use double quotes as that is the type of string that bash understands has the capabilities for string interpolation. If you use single strings, the variable won't be referenced and instead, "$myVariable" would be directly displayed

Also, you can use \" if you want to display a double quote character in your string.

2.4 Store command outputs in variables

There is this concept known as a subshell. It essentially executes a command inside a command. This allows us to store the output of a command in a variable.

2.3: 2-bash.sh

```
#!/bin/bash

files=$(ls)

echo $files
```

2.5 Environment variables

There are already some predefined variables. If you wish to see them, you can use the env command

2.4: Terminal

```
env
```

Chapter 3

Math in Bash

3.1 how to run basic math operators

For you to make basic operations like 10 + 9, you need to use the "evaluate expression" command.

3.1: 3-bash.sh

```
#!/bin/bash

#the space between value and operator is mandatory
expr 30 + 10

expr 30 - 10

expr 30 / 10
```

3.2 Wild cards

You might think that doing a multiplication is as simple as doing "expr 10 * 10" but you'd be wrong. This is because the asterisk character is a wild card. Wild cards are essentially special caracters that return specific files depending on their own criteria. For example, *.log essentially looks for all files that end in .log and returns them to the command. this means that a command might perform an operation on all files that end in .log as oposed of only doing one operation. Using '*' without anything else means to literally use everything.
So how do we solve this issue? we have to use the escape character

3.2: 3-bash.sh

```
#!/bin/bash

expr 30 \* 10
```

3.3 Math with variables

Going back to the previous chapter, we can use variables with values and perform operations

3.3: 3-bash.sh

```
#!/bin/bash

myVar=100

expr $myVar + 50
```

3.4 Arithmetic expansion

Now we can continue using expr or use the more modern and prefered way of doing math in general which is using arithmetic expansions. This don't have space constraints nor require flags.

3.4: 3-bash.sh

```
#!/bin/bash

# because the arithmetic expansion is a command itself, we must implement with a subshell
myVar=$((20 * 100))

echo $myVar
```

As we'll see later, using the subshell isn't required for if conditionals.

Chapter 4

If statements

4.1 If condition

If statements are weird in bash. First of all, bash is very space sensitive so manage with care. Then the syntax is very atypical.

4.1: 4-bash.sh

```
#!/bin/bash

myNum=200

# All spaces are mandatory!

if [ $myNum -eq 200 ]
then
    echo "the condition is true"
fi # if backwards. bruh
```

Now, you might be wondering why are we using a command flag inside our if statement? And the reason for that is because actually, the square bracket is a command itself. It's an abreviation of the test command. The -eq flag stands for equals to and expects an argument afterward. The closing square bracket is just expected by the alias definition of the test command.

4.2 A more modern approach for number comparisson

Now, if you wanted to use your typical ==, !=, ¡, etc operators,you would use a double parentheses command. This is as modern as the eyar 1997 can be labeled modern, which is not at all. So use this if you want a more normal experience.

4.2: 4-bash.sh

```
#!/bin/bash
myVar=200
if(($myVar==200))
then
    echo "myVar is equal to 200"
fi
```

4.3 Else statement

4.3: 4-bash.sh

```
#!/bin/bash
myVar=200
if(($myVar==200))
then
    echo "myVar is equal to 200"
else
    echo "myVar is not equal to 200"
fi
```

4.4 Not operator

4.4: 4-bash.sh

```
#!/bin/bash
myVar=200

# You can add an exclamatoin mark just like any other language, without the need of
    further parentheses
if((!$myVar==200))
then
    echo "myVar is not equal to 200"
else
    echo "myVar is equal to 200"
fi
```

4.5 -f flag

Since the '[]' operator is a bash command, it can interact with directories, files and such. This means we can compare agains files or directories directly. The '-f' flag is just one way to do this, where it checks wether a path exists AND is a file. if you just wanted to see if it existed as a file or directory, you'd use the -e flag.

4.5: 4-bash.sh

```
#!/bin/bash

# Does the 1-bash.sh file exist?
if [ -f ./1-bash.sh ]
then
    echo "the 1-bash.sh file exists"
else
    echo "the 1-bash.sh file doesn't exist"
fi
```

4.6 A more "practical" example

We want to check if a command exists, if it does run it, otherwise install it and then run it.

4.6: 4-bash.sh

```
#!/bin/bash

# it's good practice to use double quotes in case the path has spaces. if they do, bash
    might try to run a command afterwards leading to errors.
command="/usr/bin/htop"

# Yes, you also have to use double quotes in here for the same reason
if [ -f "$command" ]
then
    echo "$command exists"
else
    echo "$command does not exist, installing it"
    # && means 'AND' execute this other command but only if the previous one executed
    without issues
    sudo apt-get update && sudo apt-get upgrade -y && sudo apt install htop
fi

#execute command
$command
```

4.7 The better way of doing it

We used the test command to check wether a file existed in our system. However, this is a very general way of doing it, the shell has built in ways of managin this specific situation. We want to check not if a file exist but if a command exists. So we'll use a Shell builtin known as a command. Shell builtins are like any other commands but they are baked directly into the shell as oposed of being binary files. A common example of this would be the cd command. If you wish to know whether a command is a shell builtin, use the type command.

4.7: 4-bash.sh

```
#!/bin/bash

myCommand=htop

# command looks if the specified command exists. If it does, the -v flag outputs the
    direction of the command. or just the name of it if it's a shell builtin. Otherwise,
    it won't return anything
if command -v $myCommand
then
    echo "$myCommand is installed, running it"
else
    echo "$myCommand not installed, installing it"
    sudo apt-get update && sudo apt-get upgrade -y && sudo apt install $myCommand -y
fi

$myCommand
```

The reason why you wouldn't want to use which is because the which command doesn't contemplate shell buitlin commands so it might not work with every command. Also command tends to be faster since it doesn't require to open up a file and do many processes but is baked into the running process. known as the shell.

4.8 Logic operators

We already saw the AND operator. Naturally, it can also be used inside the if statement argument.

4.8: 4-bash.sh

```
#!/bin/bash

# Either one most exit with 0 for the if statement to run its inner code
if command -v top || command -v htop
then
    echo "It is possible to profile the processes"
else
    echo "no terminal tools were found to profile processes"
fi
```

4.9 Else if

Of course any well self respecting language would have the inclussion of the humble else if.

4.9: 4-bash.sh

```
#!/bin/bash

myCommand="cd"

if which $myCommand
then
    echo "$myCommand exists"
elif command -v $myCommand
then
    echo "$myCommand exists"
else
    echo "$myCommand doesn't exist"
fi
```

4.10 Ok but how does the if statement actually work?

The if statements in bash scripting don't work with your typical boolean values, they are not expecting a true nor a false. What they work with is exit codes. If a command ran successfully, it will return a 0 exit code otherwise, any other number. So 0 behaves like true, any other value behaves like false. This is the opposite of how you'd expect it to work. More on exit codes next chapter.

Chapter 5

Exit codes

5.1 Check the exit code of the previous command

you can check the exit code of previous commands using the dollar sign. If it's 0, it means success. Otherwise, it means failure. The number might give you a clue on the failure actually was about.

5.1: 5-bash.sh

```
#!/bin/bash

package=htop

sudo apt install $package

if(($?==0))
then
    echo "the installation of $package was successful"
    echo "you can find it here:"
    which $package
else
    echo "failed to install $package"
fi
```

5.2 Redirects

Redirects allow us to save the output of a command directly into a file.

5.2: 5-bash.sh

```
#!/bin/bash

package=htop

# Using a single greater than symbol would result in the specified file being overwritten
# Additionally, the output won't be displayed on the cli. It will be fully redirected*
# *It won't in some cases but we'll see that later

sudo apt install $package > package_install.log

# Using 2 greater than symbols would add the output at the end of the file.
sudo apt install $package >> package_install.log
```

Something to consider is that since the file doesn't follow a strict path, the script will create the file in your current path where you executed the script.

5.3 Careful with $?

Something you have to remember is that $? will only give you the exit code of the last command so if you want to check the first command, you have to store the value in another variable

5.3: 5-bash.sh

```
#!/bin/bash

myPath=/etc

if [ -d myPath ]
then
    echo "The $myPath directory exist"
else
    echo "The $myPath directory doesn't exist"
fi

# This will always print a 0 because it checks the exit code of the echoes which are
    always 0.
echo "The exit code is $?"
```

5.4 Manually exit

If you didn't want to store the exit value in a variable, which can get convoluted real quick, you can manually exit your script with a custom code.

5.4: 5-bash.sh

```
#!/bin/bash

echo "Hello world"
exit 199

# Anything after the exit command will be ignored.
```

5.5 The final version of the 5-bash.sh script

5.5: 5-bash.sh

```
#!/bin/bash

myPath=/etc

if [ -d $myPath ]
then
    echo "The $myPath directory exists"
    exit 0
else
    echo "The $myPath directory doesn't exist"
    exit 1
fi
```

Chapter 6

While loops

6.1 Basic while loop

6.1: 6-bash.sh

```
#!/bin/bash

myVar=1

# While the condition remains true, it will continue
while (($myVar<=10))
do
    # ++ exists but apparently it doesn't work if the var starts at 0 which defeats its purpose more often than not.
    ((myVar+=1))
done

# Because it's <=, it will make a final pass on the body making myVar = 11
echo $myVar
```

6.2 The sleep command

If we wanted to give our loop or realistically any commands in our scripts a small delay, we could use the sleep command.

6.2: 6-bash.sh

```
#!/bin/bash

while [ -f ./testfile ]
do
    echo "File exists"
    sleep 0.5
done

echo "The file was deleted at $(date)"
```

In a more realistic scenario, the delay would be higher or the implementation might be entirely different.

Chapter 7

How updates work in linux

7.1 Repositories

Lets start demystifying how linux installation works by starting with the place where our updates come from, repositories. **Repositories** are servers that contain software that is constantly being updated my the mantainers and where we can download our updates from.

There is also the concept of **mirrors** which are literal copies of the repositories as they have the same content (thus the name mirrors) but are located in different places. This is useful because you can chose mirrors that are closer to you to improve download speeds

If you want to see where the repository list is located in your computer, you can check /etc/apt/source.list.d/

7.2 Starting the workflow: apt update

Each distro has their own way of updating software but we'll be looking at debian based distros for this one. The first thing we do is to "**update**":

7.1: Terminal

```
apt update
```

What "apt update" does is that it checks the list of repositories and downloads a type of metadata known as **indexes**. These indexes contain data that apt can later use to see if there is a pending update for a software, how to solve dependencies, where specifically do we install the files from (inside the repository path), etc ...

You see, The repositories are divided in to main paths, one known as "dists/" which contains metadata. Using metadata is also usefull because now we don't have to check entire gigabytes of files to see if there is an update available. Then we have the "pool/" path which contains the actual files to be downloaded.

You can check the indexes at /var/lib/apt/lists. Usually, the files are stored in compressed files.

7.3 Actually installing: apt upgrade

Now we move into the installation workflow. "apt upgrade" starts by checking those indexes and loading them into memory. Then it loads some sort of metadata of currently installed packages into memory as well which can be found at /var/lib/dpkg/status. This means that we now can compare the metadata of the repositories vs the ones corresponding to locally installed packages to see which require updates.

Something to consider is that their installation order must follow a dependency order because we cannot install a package that depends on another one. Because of this, apt builds a dependency graph, specifically a **Directed Acyclic Graph (DAG)**. DAGs are essentially like any other graph that have connections with directions, and are acyclic which means that no direction can result in the creation of a loop.

Once we know which software must be updated and in which order, we can start downloading the packages. All packages are downloaded into a cache directory at /var/cache/apt/archives/. This is so if there was to be an internet outage or a package is found corrupt, the interruption wouldn't reset the entire progress back to 0. This implies that the cache is also checked before doing the installation.

Finnally, we move into the actual installation. Apt, isn't responsible for it since it cannot process .deb files so it calls dpkg to handle that. **dpkg** is the package installer for .deb files. It will install all files in the cache respecting the order set by the DAG. We won't cover how dpkg works, for now...

7.4 The alternative: apt dist-upgrade

Sometimes, a package will need a new dependency that wasn't contemplated. Apt by default won't add any new packages on an update. Because of this, the packages will be labeled as "held back". To force apt to install new packages and resolve all sorts of dependecy issues it my encouter, you can use "apt dist-upgrade".

Chapter 8

Update script

8.1 A script for all distros

Distro might have different update mechanisms. If you deal with multiple updates, this script can be a one fits all solution.

8.1: 8-bash.sh

```
#!/bin/bash

if [ -d /etc/pacman.d ]
then
    # for archzzz
    sudo pacman -Syu
elif [ -d /etc/apt ]
then
    # for debian BASED!!1!! literally
    sudo apt update && sudo apt upgrade
fi
```

8.2 A better way of writing the script

the script relies on the specific software to be installed. However there can be nutheads that manage to install pacman in debian based distros. For this nutheads, we need to check the distro installed opposed to the update manager.

8.2: 8-bash.sh

```
#!/bin/bash

release_file=/etc/os-release
# we'll see what grep and other commands do after this chapter
if grep -q "Arch" $release_file
then
    sudo pacman -Syu
elif grep -q "ubuntu" $release_file
then
    sudo apt update && sudo apt upgrade
fi
```